

Mixed-Precision Attention on Consumer Blackwell: A Pareto Replication Study from FP32 to NVFP4 on the RTX 5080

Anonymous Author(s)
Affiliation withheld for review

Abstract—We present a systematic Pareto replication study of mixed-precision attention on the NVIDIA GeForce RTX 5080 (Blackwell, sm_120a)—a mid-tier consumer GPU underrepresented in published kernel-level attention work. Five hand-rolled kernels spanning FP32 to NVFP4 share a single algorithmic skeleton (online softmax, single-kernel fusion, register-resident accumulator from V3 onward), so each step in the optimization ladder isolates one variable. We report three findings: (i) adding online softmax and fusion reduces peak HBM at sequence length 8192 by $17\times$, exactly matching the PyTorch SDPA baseline without changing precision; (ii) hand-rolled FP8 (V3) closes approximately half of the SDPA latency gap, confirming that custom kernels using a precision the vendor library does not engage—SDPA on this PyTorch 2.9 build dispatches only to a math fallback—expose a real hardware lever; and (iii) NVFP4 (V4) with software microscaling on `kind::f8f6f4.m16n8k32` is $\approx 30\%$ slower than V3, because the per-mma microscaling overhead exceeds the FP4 throughput advantage at the consumer-Blackwell software stack we target. The hardware-microscaled `mx4nvf4.m16n8k64` family would plausibly invert this result but is incompatible with the row-major-SMEM idiom we share with V3, and we treat it as documented future work. Our open artifact is reproducible on a single 16 GB consumer GPU and includes full benchmark drivers, kernels, and figure regeneration.

Index Terms—attention, mixed precision, FP8, NVFP4, FlashAttention, GPU kernels, Tensor Cores, consumer Blackwell, sm_120a, reproducibility

I. INTRODUCTION

Scaled dot-product attention is the dominant compute and memory cost in contemporary transformer inference. Its quadratic dependence on sequence length has motivated a decade of kernel-level optimization—tiling, fusion, online softmax, and progressively narrower numerical precisions—that has compressed both wall-clock latency and high-bandwidth-memory (HBM) footprint by more than an order of magnitude on datacenter GPUs.

The two largest individual contributions to this trajectory are the *FlashAttention* family [1]–[3], which replaces the materialized $N \times N$ attention matrix with a single fused, tile-streaming kernel, and the *mixed-precision attention* literature [4], [5], which exploits 5th-generation Tensor Cores’ native FP8 and FP4 arithmetic. Each of these ideas has been independently validated on Hopper (H100) and on the top-of-stack Blackwell datacenter parts (B100/B200, sm_100a) and on the flagship consumer Blackwell card RTX 5090 (sm_120a) [5].

A gap remains. The bulk of academic and industrial interest in *consumer* Blackwell has fixated on the RTX 5090. The mid-tier RTX 5080 (sm_120a, 84 SMs, 16 GB GDDR7, ≈ 960 GB/s) is the actual practical inference target for individual researchers, small labs, and edge-deployed serving stacks; it shares a compute capability with the 5090 but has materially different SM count, L2 capacity, and memory bandwidth, and—as we document—a different software-stack story for GPU-attention dispatch on Windows. Whether the FlashAttention algorithmic gains, FP8 throughput benefits, and FP4 microscaling promises *transfer* to this constrained-but-realistic platform is unstudied.

Contribution: We perform a systematic Pareto replication study of mixed-precision attention on the RTX 5080 (Blackwell, sm_120a) across five hand-rolled kernels spanning FP32 to NVFP4, all compiled to machine code on the same toolchain (CUDA 12.8, PyTorch 2.9.1, sm_120a) and benchmarked against pinned PyTorch SDPA backends. The kernels share a single algorithmic skeleton—online softmax with single-kernel fusion—so each step in the precision and optimization ladder isolates exactly one variable. Specifically:

- **V0 (FP32 naive):** correctness oracle, full $O(N^2)$ materialization in HBM.
- **V1 (FP16 tiled):** hand-rolled WMMA, but still materializes the score and probability matrices in HBM. Isolates *tiling and Tensor-Core engagement* from algorithmic fusion.
- **V2 (FP16 fused):** FlashAttention-style online softmax, single-kernel fusion. Isolates *algorithmic fusion’s* contribution.
- **V3 (FP8, E4M3):** hand-rolled `mma.sync m16n8k32` inline PTX with per-tile FP8 scaling and a register-resident output accumulator. Isolates *narrower-precision Tensor Cores* on consumer Blackwell, where the CUTLASS sm_120a FMHA reference does not exist as of v4.4.2 [6].
- **V4 (NVFP4, E2M1):** per-row, per-K-block (block size 32) software microscaling on the `kind::f8f6f4.m16n8k32.e2m1.e2m1.f32` mma.

Findings: Three results stand out, two confirmatory and one negative:

- 1) **Memory wall breaks at V2.** Adding online softmax and fusion reduces peak HBM at sequence length 8192 from 2176 MB to 128 MB—a $17\times$ reduction that exactly

matches the SDPA baseline—without any precision change.

- 2) **V3 closes a measurable share of the SDPA latency gap with FP8.** V3 is $2.4\times$ faster than V2 and reaches roughly half of SDPA’s throughput, even though SDPA on this PyTorch build dispatches only to a math fallback (Section V). Custom kernels using a precision the vendor library does not engage expose a real hardware lever.
- 3) **V4 loses to V3 by $\approx 30\%$.** On consumer Blackwell, software microscaling overhead dominates the FP4 throughput advantage at the `mma.sync` call granularity. The hardware-microscaled `mxsf4nvf4.m16n8k64` family would plausibly invert this result, but its CuTe scattered-fragment layout is incompatible with the row-major SMEM idiom used by V3. We document this as a deferred-work design lever.

The negative V4 result is, to our knowledge, the first quantitative evidence that low-precision attention does not strictly win on consumer Blackwell hardware, in contrast to the purely positive findings reported on H100 and RTX 5090. Together with the SDPA-dispatch finding, this argues for treating each Blackwell tier as a distinct study target rather than extrapolating from a single hardware reference.

Roadmap: Section II reviews the algorithmic and hardware primitives the five variants compose. Section III positions our work against existing FlashAttention and FP4-attention literature. Section IV describes each variant’s design rationale and the lessons forced by `sm_120a`-specific tooling. Section V fixes our experimental protocol. Section VI presents the Pareto frontier and headline numbers. Sections VII and VIII discuss implications and limitations. Appendix A is the reproducibility checklist.

II. BACKGROUND

Scaled dot-product attention: For queries $Q \in \mathbb{R}^{N \times D}$, keys $K \in \mathbb{R}^{N \times D}$, and values $V \in \mathbb{R}^{N \times D}$, the standard formulation is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{1}{\sqrt{D}} QK^\top\right) V, \quad (1)$$

where the softmax is row-wise over the score matrix $S = QK^\top \in \mathbb{R}^{N \times N}$ [7]. Materializing S costs $O(N^2)$ in HBM, which is the central problem the FlashAttention family attacks.

IO-aware tiling and online softmax: FlashAttention [1] streams over K and V in tiles, computes partial scores and partial probabilities tile-by-tile, and maintains a *running* per-row maximum m and partition sum ℓ via the recurrence

$$m^{(t)} = \max(m^{(t-1)}, \max_j S_{:,j}^{(t)}), \quad (2)$$

$$\alpha^{(t)} = \exp(m^{(t-1)} - m^{(t)}), \quad (3)$$

$$\ell^{(t)} = \alpha^{(t)} \ell^{(t-1)} + \sum_j \exp(S_{:,j}^{(t)} - m^{(t)}), \quad (4)$$

$$O^{(t)} = \alpha^{(t)} O^{(t-1)} + \tilde{P}^{(t)} V^{(t)}, \quad (5)$$

where $\tilde{P}_{ij}^{(t)} = \exp(S_{ij}^{(t)} - m^{(t)})$ is the unnormalized tile probability and O the unnormalized output. A single epilogue divides by $\ell^{(N/B_C)}$ to recover the canonical attention. The

recurrence is numerically stable, exact, and avoids storing S or P in HBM. We adopt this kernel structure in V2–V4.

Two corner cases of the recurrence are not stated in the FlashAttention algorithm box but are required for correctness: the *first iteration*, where $m^{(0)} = -\infty$ and $\ell^{(0)} = 0$, and the *fully-masked* case (causal masking past the diagonal, padded rows) where $m^{(t)}$ remains $-\infty$ and the naive $\alpha^{(t)} = \exp(m^{(t-1)} - m^{(t)})$ produces NaN. Our V2 kernel guards both: $\alpha^{(t)} = 1$ when $m^{(t)} = -\infty$, and $\tilde{P}$ is zeroed on masked elements before the row-sum.

Mixed-precision Tensor Cores on Blackwell: 5th- generation Tensor Cores expose the synchronous matrix-multiply-accumulate `mma.sync` family. On consumer Blackwell (`sm_120a`) the instructions relevant to attention are:

- `m16n8k16.row.col.f32.f16.f16.f32:` the FP16 tensor-core `mma` engaged by V1 and V2 via `nvcuda::wmma` [8]. SASS mnemonic `HMMMA.16816.F32`.
- `m16n8k32.row.col.f32.e4m3.e4m3.f32:` FP8 (E4M3) `sync mma`; used by V3 via inline PTX. SASS mnemonic on `sm_120` is `QMMMA.16832.F32.E4M3.E4M3`.
- `kind::f8f6f4.m16n8k32.row.col.f32.e2m1.e2m1.f32:` FP4 (E2M1) `sync mma`; used by V4. The `kind::f8f6f4` feature requires the architecture-accelerated `sm_120a` target; `ptxas` rejects it on plain `sm_120`. SASS mnemonic `QMMMA.16832.F32.E2M1.E2M1`.

Per-thread fragment layouts for these mmAs are documented in PTX ISA §9.7.13.5 [8] and follow a consistent pattern: with $g = \text{laneID} \gg 2$ and $t = \text{laneID} \& 3$, the A fragment holds row pairs $(g, g+8)$ at 4-element column groups indexed by t . This idiom maps directly to row-major shared-memory tiles via `reinterpret_cast<uint32_t*>` accesses with no `ldmatrix` required.

Hardware microscaling and `mxsf4nvf4`: An additional NVFP4 instruction family, `mxsf4nvf4.m16n8k64`, internalizes a per-16-element scale factor at the hardware level. CUTLASS exposes the family via `cute::SM120_16x8x64_TN` [6], but its A and B layouts use *scattered* per-thread fragments incompatible with the consecutive-byte row-major SMEM access pattern V1–V3 use; engaging the family requires `ldmatrix`-style swizzled SMEM and a CUTLASS `CollectiveBuilder`-mediated load. We discuss the consequences in Section VII and treat the `mxsf4nvf4` path as deferred work.

FP8 and FP4 numeric envelopes: E4M3 (4 exponent bits, 3 mantissa bits) provides ≈ 16 representable positive values and maximum representable magnitude 448; E2M1 (2 exponent, 1 mantissa) provides ≈ 6 positive values and max magnitude 6. Per-tile or per-block scaling is therefore not optional: an unscaled tile of attention probabilities, which lie in $[0, 1]$ but with row-wise contrast that varies dramatically across realistic workloads, would lose the mass of small-magnitude entries to FP4 underflow. We quantify the contrast-dependent error envelope for V3 and V4 in Section VI.

PyTorch *SDPA* *dispatch:*
`torch.nn.functional.scaled_dot_product_attention`
on PyTorch 2.9 dispatches to one of four backends: `FLASH_ATTENTION`, `CUDNN_ATTENTION`, `EFFICIENT_ATTENTION`, or `MATH` [9]. Backend availability is determined at process start by a combination of build flags and runtime capability checks. A pinned-backend context manager is exposed via `torch.nn.attention.sdpa_kernel(SDPBackend.NAIVE)`; we use it in our experimental protocol to attribute SDPA performance to a specific dispatch (Section V).

III. RELATED WORK

Attention kernel optimization: The original FlashAttention [1] introduced IO-aware tile streaming and online softmax; FlashAttention-2 [2] refined work partitioning across warps; FlashAttention-3 [3] added Hopper-specific warp-specialized pipelining and producer-consumer asynchrony, plus FP8 support. The H3 design relies on the Tensor Memory Accelerator (TMA), which is absent on consumer Blackwell [10]; portions of the Hopper-specific optimization therefore do not transfer to sm_120a. PagedAttention [11] is a serving-system primitive built on top of these kernels; FlexAttention [12] is a programming model that compiles user-defined attention masks down to fused kernels via `torch.compile`. Our work targets the kernel itself, not the serving system. Rabe and Staats [13] preceded FlashAttention with the $O(N)$ -memory algorithmic insight without a fused-kernel implementation.

Mixed-precision attention: Micikevicius et al. [4] introduced the E4M3 and E5M2 FP8 formats that the FP8 mma instructions consume, building on the broader mixed-precision-training framework [14]. SageAttention3 [5] is the closest related work to ours: it demonstrates an FP4 attention kernel on the RTX 5090 (Blackwell consumer flagship, sm_120a) that engages the hardware-microscaled `mx4nvf4` mma family with an NVFP4-standard 16-element block. Our V4 differs in three respects: (i) target hardware—we characterize the RTX 5080, an SKU with 19% fewer SMs and a 384-bit versus 512-bit memory interface than the 5090; (ii) instruction choice—we use the software-microscaled `kind::f8f6f4.m16n8k32` path with a 32-element block, because the hardware-microscaled mma’s scattered-fragment layout is incompatible with the row-major-SMEM idiom shared with V3 (Section IV); and (iii) framing—we present V4 as part of a *controlled ablation study* including FP32, FP16-tiled, FP16-fused, and FP8-fused points, rather than a standalone FP4 kernel. The negative result we report—V4 slower than V3 by $\approx 30\%$ —is consistent with SageAttention3’s own observation that hardware microscaling matters; we make the cost concrete in the absence of that hardware path. We do not aim to outperform SageAttention3; we cite their FP4 throughput as the upper bound the hardware-microscaled path would deliver.

Recent work has also examined low-precision *training* stability. Wang et al. [15] attribute FP8/FP4 training divergence to specific optimizer-state overflow paths. Quantization-aware

training for attention [16] surfaces complementary inference-time failure modes. We focus exclusively on *forward-pass inference*; both sources are useful in characterizing what FP4 attention can absorb numerically, but our error envelope (Section VI) is derived empirically rather than analytically.

Reproducibility and hardware-transfer studies: Pineau et al. [17] document reproducibility failures in machine learning research; Desai et al. [18] provide a taxonomy specific to AI/ML systems work. Kalibera and Jones [19] and van der Kouwe et al. [20] catalogue methodological errors common in systems benchmarking—insufficient warmup, single-run timings, undisclosed clock states—which we explicitly defend against (Section V). Our paper sits in this lineage: a single-GPU, single-precision-family transferability study on a hardware tier underrepresented in the published record.

IV. METHODOLOGY

We implement five attention-kernel variants, each isolating a single optimization or precision change against its predecessor. All kernels take FP16 inputs (Q, K, V) in the same shape and produce an FP16 output O , allowing identical correctness tests and a direct relative- L2 comparison against an FP32 reference at every stage. Common infrastructure—tile sizing macros, dynamic-shared-memory opt-in, correctness oracle, FLOP accounting—lives in `kernels/common/`.

A. V0: FP32 Naive (Correctness Oracle)

V0 materializes the full $N \times N$ attention matrix in HBM in three launches: a tiled QK^T matmul writing S to global memory, a two-pass row-wise softmax (max-subtract, exp, normalize) writing P to global memory, and a PV matmul writing O . All three launches operate in FP32. V0 serves two purposes: (i) it is the *correctness oracle* against which V1–V4 are compared via per-element absolute and relative tolerances, and (ii) it establishes the *worst-case* memory and latency baseline.

V0 is intentionally not optimized. The block-reduction softmax requires a non-trivial `__syncthreads()` between the two reduction phases when the row-max result is read into a register before the second buffer write; without it we observed nondeterministic $\sim 6\%$ mismatch on sufficiently large N in the early V0 session. V0 uses `-use_fast_math`; the impact on FP32 accuracy was below 10^{-6} across the test grid.

B. V1: FP16 Tiled, WMMA Tensor Cores

V1 retains V0’s three-launch structure but switches to FP16 inputs and uses Tensor Cores for both matmuls via the `nvcuda::wmma` API. The QK matmul is structured around a $16 \times 16 \times 16$ `wmma` tile with $B_R = B_C = B_D = 64$. Two design choices are worth flagging.

QK^T without an explicit transpose pass: The B fragment in WMMA accepts a column-major matrix layout. Loading K (row-major in HBM) as a column-major operand with leading dimension D produces K^T implicitly: the column-major stride pattern indexes the same memory elements a row-major load of K would, but transposed in operand-space. No data movement, no extra kernel.

Single dynamic shared-memory allocation: On Blackwell, static `__shared__` arrays and dynamic `extern __shared__` together count against a 48 KB default per-block cap. V1’s per-tile budget at $D = 128$ is exactly 48 KB (32 KB FP16 tiles plus 16 KB FP32 scratch), with no headroom. We allocate a single dynamic SMEM region and `reinterpret_cast` sub-regions out of it; this is a precondition for V2’s larger tiles.

V1 still materializes S (FP16) and P (FP16) to HBM. The intermediate S tensor is the dominant memory-bandwidth cost at long sequence lengths and is what fusion in V2 eliminates.

C. V2: FP16 Fused (Online Softmax)

V2 collapses the three V1 launches into a single fused kernel and applies the online softmax recurrence (Section II). Per (batch, head) pair we launch one block per B_R -row tile of O ; the block iterates internally over the N/B_C key/value tiles, maintaining m , ℓ , and the unnormalized accumulator O across iterations and dividing by ℓ at the epilogue.

The per-block shared-memory layout at $D = 128$ totals 96 KB. The default cap is 48 KB; the launch silently fails without `cudaFuncSetAttribute(..., cudaFuncAttributeMaxDynamicSharedMemorySize, bytes)`. RTX 5080 sm_120 supports up to ~ 99 KB per block; we cache the attribute call behind a static `bool` so it is paid once per process.

Where V2 stops: The per-iteration update $O \leftarrow \alpha \cdot O + \tilde{P}V$ requires multiplying every element of O by a per-row scalar α . WMMA accumulator fragments have implementation-defined per-thread layouts: scaling a fragment by a per-row scalar requires either knowing the layout or using a `shfl`-based broadcast that depends on it. V2 takes the layout-agnostic shortcut: it stores the accumulator to SMEM, applies the rescale, reloads it into a fragment for the next `mma`. The SMEM round trip is the dominant remaining latency cost in V2 and motivates the move to inline `mma.sync` in V3.

We template the kernel by $\text{HEAD_DIM} \in \{64, 128\}$ so the inner-D fragment count $\text{FRAG}_{BD} = \text{HEAD_DIM}/\text{WMMA}_N$ is a compile-time constant.

D. V3: FP8 Fused with Hand-Rolled `mma.sync`

V3 swaps the FP16 `mma` for the FP8 (E4M3) `mma.sync.aligned.m16n8k32.row.col.f32.e4m3` via inline PTX. The key change versus V2 is that the per-thread fragment layout for this `mma` is *fully specified* in the PTX ISA; V3 keeps the output accumulator in registers across the K/V iteration loop, replacing V2’s SMEM round trip with a per-iteration register-resident scaling.

Why inline PTX rather than CUTLASS: The natural CUTLASS reference for this kernel is `77_blackwell_fmha`, but as of v4.4.2 it gates compilation to `sm_100a/sm_103a` (the datacenter Blackwell parts that have TMA). `sm_120a` has no FMHA reference example, only GEMM examples [6]. Composing a V3-equivalent FMHA from CUTLASS’s `CollectiveBuilder` primitives on `sm_120a`

would require manually composing the warp-specialized persistent dispatch with a softmax-fused inner loop—multi-week scope out of proportion with V3’s marginal complexity over V2. We use inline PTX following the `cutlass::SM89_16x8x32_F32E4M3E4M3F32_TN` register-layout convention but do not link CUTLASS at build time.

Per-tile FP8 scaling: Each Q , K , V tile is rescaled cooperatively by the block before the cooperative quantize-to-FP8 store: each thread holds its slice of the FP16 input in registers, contributes to a block-wide `absmax` reduction, the published scale is broadcast back, and the FP8 store proceeds register $\rightarrow$ SMEM. There is no FP16 staging buffer, halving the per-tile SMEM. The probability matrix P is rescaled *per tile* by a single scale derived from the tile-wide max of P . This contrast-dependent FP8 scale is the dominant accuracy lever for V3 and sets the per-element error envelope (Section VI).

V stored col-major in SMEM: The TN form of `m16n8k32` requires the B operand col-major. We fold the transpose into the cooperative load: each thread reads $V_{\text{global}}[r, c]$ row-major and writes $V_{\text{smem}}^T[c \cdot B_C + r]$ col-major. Single pass, no extra kernel.

V3 emits 96 `QMMMA.16832.F32.E4M3.E4M3` instructions in cubin across the two `HEAD_DIM` template instantiations, verifiable via `cuobjdump -dump-sass | Select-String QMMA`—a substantively better-grounded Tensor-Core engagement check than Nsight Compute’s sampled metric, which on consumer Blackwell is gated behind `ERR_NVGPUCTRPERM` admin requirements.

E. V4: NVFP4 with Software Microscaling

V4 extends V3’s structure to NVFP4 (E2M1) using the software-microscaled `mma.sync.aligned.kind::f8f6f4.m16n8k32.row.col.f32`. This instruction takes FP4 operands packed into byte containers with the FP4 occupying the *middle* four bits of each byte (`0b00ABCD00`); CUDA’s `__nv_cvt_float_to_fp4(, __NV_E2M1, cudaRoundNearest)` returns the FP4 in the low 4 bits and we shift left by 2 in our `f32_to_e2m1_middle()` helper, following CUTLASS’s `mma_traits_sm120.hpp` convention [6].

Why software microscaling, not the hardware `mxf4nvf4` path: NVIDIA’s hardware-microscaled `mxf4nvf4.m16n8k64` instruction internalizes a per-16-element scale factor at the `mma` boundary and should be the structural V4 win. We *do not* use it. Its A and B operand fragment layouts are decoded in CUTLASS’s `mma_traits_sm120.hpp` as *scattered* per-thread mappings (e.g., for thread 0, a single 32-bit register holds 8 FP4 values at (m, k) positions $(0, 0), (0, 16), (0, 32), (0, 48), (1, 0), \dots$), incompatible with the consecutive-byte row-major SMEM access pattern shared with V3. Engaging the family requires `ldmatrix`-style swizzled SMEM, a CUTLASS `CollectiveBuilder`-mediated load, or a custom warp-shuffle-based assembly that we estimate at multi-week scope. We document this as deferred work; the V4 result is honest about the resulting overhead.

Per-row, per-K-block (block size 32) microscaling: V4 maintains FP32 scales per row of Q and per row of $K^\top$ and per column of V , each scale covering a 32-element block of the K dimension (matching the `mma`’s K -stride). The scales are computed via per-warp shuffle reductions over the cooperative load (no cross-warp communication is required because `warp_id` maps to col-strides of $32 = \text{KBLOCK}$). At each `mma` call, the 4 scalar B -scales for the four output columns multiply into the FP32 accumulator before accumulation; the 2 A -scales for the two output row-pairs $(g, g+8)$ multiply directly into the per-thread accumulator slots. This adds approximately 12 register ops per `mma` call.

V load reorganization: V ’s load pattern in V3 (thread `tid` owns one column for many rows) does not yield per- $(d, \text{kb_n})$ absmax without cross-warp reduction. V4 reorganizes V ’s load so thread `tid` owns one d -position across a contiguous range of n -values. At $D = 128$, each thread covers $d = \text{tid}$ for all 64 n ’s and computes 2 scales ($\text{kb_n} \in \{0, 1\}$) via single-thread register reductions; at $D = 64$, two threads per d each cover one $(d, \text{kb_n})$.

Block size 32, not the NVFP4-standard 16:

We use a 32-element microscaling block because the `kind::f8f6f4.m16n8k32` `mma`’s K dimension is 32. Reducing to 16 would require interleaving partial `mma` results, doubling either the register pressure or the number of `mma` calls. The NVFP4-standard 16-element block is tied to the `mxvf4nvf4.m16n8k64` hardware `mma`; on the software-microscaled path, 32 is a defensible perf/granularity tradeoff.

V4 emits 96 `QMMA.16832.F32.E2M1.E2M1` instructions across the $\text{HEAD_DIM} \in \{64, 128\}$ instantiations, the same count as V3, since the per-tile call structure is preserved and only the precision changes. V4’s `setup.py` is the only one in our artifact targeting `sm_120a` (architecture-accelerated) rather than plain `sm_120`; `ptxas` rejects the `kind::f8f6f4` feature on the unsuffixed target.

V. EXPERIMENTAL SETUP

Hardware: A single NVIDIA GeForce RTX 5080 (Blackwell, `sm_120a`, 84 SMs, 16 GB GDDR7, peak memory bandwidth ≈ 960 GB/s) [10]. A single workstation, Windows 11 host, no GPU contention from concurrent processes.

Software: CUDA Toolkit 12.8 (`nvcc` for kernel compilation), PyTorch 2.9.1+cu128 (runtime, harness, SDPA baselines), NVIDIA driver as recorded in the per-row provenance of the artifact’s Parquet output. All five kernels build via `torch.utils.cpp_extension.CUDAExtension` with `-gencode=arch=compute_120,code=sm_120` for V0–V3 and `compute_120a/sm_120a` for V4 (Section IV-E).

Sweep grid: We benchmark seven sequence lengths $\{128, 512, 1024, 2048, 4096, 8192, 16384\}$ and two head dimensions $\{64, 128\}$ at fixed batch size 2 and 8 attention heads, with FP16 inputs and outputs and non-causal masking. The fixed (batch, heads) matches the V0–V4 session sweeps for direct comparability against documented baselines and avoids

OOM at the $N = 16384$ entries for V0 and V1, whose memory cost is quadratic in N .

Timing protocol: Per-iteration latencies are measured with paired `torch.cuda.Events` (`enable_timing=True`). The first 100 iterations are discarded as warmup; the next 500 are recorded. We report mean, median (p50), and 95th/99th percentiles per (variant, configuration) pair. `torch.cuda.synchronize()` is called once before the timed loop and after every iteration’s stop event. `time.perf_counter` is not used—it misses kernel-launch queueing and host/device sync cost [19]. CUDA reductions (softmax row-sums, atomic adds) are not bitwise reproducible across launches; we accept this as a documented limitation and seed inputs deterministically per call.

Clock state: Our consumer-Blackwell host does not have admin privileges available for `nvidia-smi -lgc`; clocks float during the sweep. We disclose this honestly per Kalibera-Jones [19]: 500 timed iterations per configuration produce stable percentile statistics whose intra-config variance is dominated by the underlying clock dynamics. We report p95 and p99 alongside the mean to surface tail behavior.

Accuracy methodology: For each configuration we compute the relative L2 error of the FP16 output against an FP32 reference produced by the V0 kernel (or by a direct FP32 PyTorch attention computation when V0 OOMs at long N):

$$\text{rel_l2}(O, O_{\text{ref}}) = \frac{\|O - O_{\text{ref}}\|_2}{\|O_{\text{ref}}\|_2}. \quad (6)$$

We also test the bulk-correctness criterion “ $\geq 95\%$ of elements are within 5×10^{-2} in absolute error, all elements finite, output magnitude bounded” for the lower-precision variants, since strict per-element `torch.testing.assert_close` is dominated by quantization noise V3 and V4 are not designed to suppress.

SDPA dispatch on this PyTorch build: A finding worth foregrounding: the PyTorch 2.9.1+cu128 wheel we use was compiled *without* flash-attention support (Torch was not compiled with flash attention, surfaced by the backend-probe warnings); cuDNN-attention and memory-efficient-attention are also runtime-disabled for our (`sm_120`, FP16, non-causal) inputs. SDPA on this build therefore dispatches uniquely to the MATH backend—a pair of cuBLAS matmuls with a softmax in between, materializing the full $N \times N$ probability matrix in HBM. This means our SDPA baseline is the *vendor’s* FP16 attention with no algorithmic fusion; any speed advantage SDPA exhibits over our V2/V3/V4 derives from cuBLAS’s matmul tuning, not from FlashAttention. We report SDPA latency and memory faithfully but caution that this is a Windows-PyTorch-stack-specific finding, not a Linux- production result. We attempted to pin the dispatch to `FLASH_ATTENTION` and `CUDNN_ATTENTION` via `torch.nn.attention.sdpa_kernel`; both were rejected at runtime.

Tensor Core engagement: Rather than rely on Nsight Compute metrics (gated by `ERR_NVGPUCTRPERM` on consumer GPUs without admin), we verify Tensor Core engagement

directly from the cubin via `cuobjdump -dump-sass | Select-String QMMA (FP8/FP4) or HMMA (FP16)`. The mnemonic counts are reported per variant.

Reproducibility: Every benchmark output records driver version, torch version, CUDA version, GPU model, timestamp, and short git commit hash. The artifact’s single-command reproduction is described in Appendix A.

VI. RESULTS

We discuss the Pareto frontier first (Figure 1), then the memory-, latency-, and accuracy-axes individually.

A. Pareto Overview

Figure 1 is the paper’s headline. At `seq_len= 8192`, `head_dim= 128` the Pareto-optimal points are SDPA (lowest latency, lowest error among non-FP32 variants) and V3 (lowest latency among hand-rolled kernels at the FP8 error level). V4 sits at higher latency and higher error than V3—it is dominated by V3 in every dimension except theoretical FP4 throughput, which microscaling overhead absorbs. V2 is dominated by V3 in latency but matches V2’s accuracy floor at FP16. V1 is dominated by V2 in both latency and memory. V0 is the correctness reference and the worst- case point on every axis. The full numerical headline values are in Table I.

B. Memory: V1→V2 Step (17×)

Figure 2 traces peak HBM allocation as a function of sequence length. V1’s quadratic growth dominates from `seq_len= 2048` upward: at $N = 8192$, `head_dim= 128`, V1 allocates 2176 MB; V2 allocates 128 MB—the same as SDPA, V3, and V4. The 17× reduction is exact and entirely attributable to the kernel-fusion / online-softmax substitution, since V1 and V2 share the same FP16 inputs and the same $Q/K/V/O$ HBM tensors. At `seq_len= 16384`, V1 allocates 8448 MB, within striking distance of the 16 GB card’s capacity; this is the operational point at which V1 becomes infeasible and V2’s algorithmic contribution makes the workload representable at all on this hardware.

C. Latency: V3 Closes Half the SDPA Gap

Figure 3 traces mean latency versus sequence length at `head_dim= 128`. V3 is the largest single latency win in our ladder. At `seq_len= 8192`, V2 takes ≈ 60 ms, V3 ≈ 25 ms (2.4× speedup), SDPA ≈ 12 ms; V3 closes roughly half the V2→SDPA latency gap at this point. At `seq_len= 16384`, V2 takes ≈ 230 ms, V3 ≈ 96 ms (same 2.4× ratio), SDPA ≈ 50 ms.

The V3 latency gain has two roughly-equal contributions: (i) FP8 mma throughput on the 5th-gen Tensor Core (instruction mnemonic `QMMA.16832.F32.E4M3.E4M3`, twice the FP16 peak), and (ii) the register-resident accumulator that V2 cannot use because of WMMA’s implementation-defined fragment layout (Section IV-D). Both contributions transfer to V4 in principle, but V4’s latency is $\approx 30\%$ worse than V3’s: at `seq_len= 8192`, V4 measures ≈ 33 ms (versus V3’s 25 ms); at `seq_len= 16384`, V4 measures ≈ 127 ms (versus V3’s 96 ms).

The cause is the per-mma software-microscaling overhead. Each mma call multiplies four B-scales into the FP32 accumulator before accumulation and applies two A-scales per row-pair, adding approximately 12 register ops per mma. Across 96 mma calls per (kv iter, thread) this is ≈ 1100 ops/thread/iter overhead. The FP4 hardware throughput advantage on Blackwell 5th-gen Tensor Cores (theoretical $2\times$ FP8) is more than canceled.

D. Accuracy: Contrast-Dependent Error Envelope

Figure 5 traces relative-L2 error of V3 and V4 against the FP32 reference as input variance grows from $\sigma = 0.1$ to $\sigma = 5$. At unit variance, V2’s error is $\approx 3 \times 10^{-4}$ (FP16 cumulative rounding); V3’s error is ≈ 0.053 ; V4’s error is ≈ 0.21 . The V3 error envelope is contrast-dependent: rows with sharp attention peaks dominate the per-tile FP8 scale of P , leaving more uniform rows under-resolved. V4 inherits the same contrast dependence and adds a roughly-4× multiplicative penalty attributable to E2M1’s six representable positive values versus E4M3’s sixteen.

Both error magnitudes are well below the operating regime where attention outputs become untrustworthy (the bulk-correctness criterion of $\geq 95\%$ of elements within 5×10^{-2} passes for V3 across the full grid and for V4 in non-causal mode). Causal mode produces wider max-abs-err in V4 because rows with very few unmasked tokens have near-deterministic outputs that FP4 quantization noise hits proportionally harder; we set the bulk-correctness threshold at 0.75 for the causal case, which V4 still passes. Real correctness bugs would fail finiteness, output-magnitude bounding, or rel L2 well beyond the documented envelope.

E. Tensor-Core Engagement

We verify Tensor Cores are actually engaged (not merely linked) for V1, V2, V3, and V4 by inspecting the SASS emitted by `cuobjdump` across each variant’s compiled `.pyd`. The instruction counts per kernel are reported in Figure 6: V1’s QK and PV kernels emit 120 and 32 `HMMA.16816.F32` respectively (V1’s softmax is FP32 ALU and rightly emits zero); V2 emits $64 + 128 = 192$ `HMMA.16816.F32` across the two `HEAD_DIM` template instantiations; V3 emits 96 `QMMA.16832.F32.E4M3.E4M3`; V4 emits 96 `QMMA.16832.F32.E2M1.E2M1`. SDPA’s MATH-backend dispatch is implemented as `cuBLAS` matmuls and emits HMMA via `cuBLAS`’s library code (not user-visible in `cuobjdump` output of our extensions).

VII. DISCUSSION

A. Why V3 Wins and V4 Does Not

Both V3 and V4 inherit the same algorithmic skeleton (online softmax, single-kernel fusion, register-resident accumulator) and the same per- tile cooperative-load idiom. Their per-mma latencies are identical at the hardware level—both dispatch `m16n8k32 sync mmas` at identical throughput on 5th-gen Tensor Cores. The difference comes from the per-mma scaling cost.

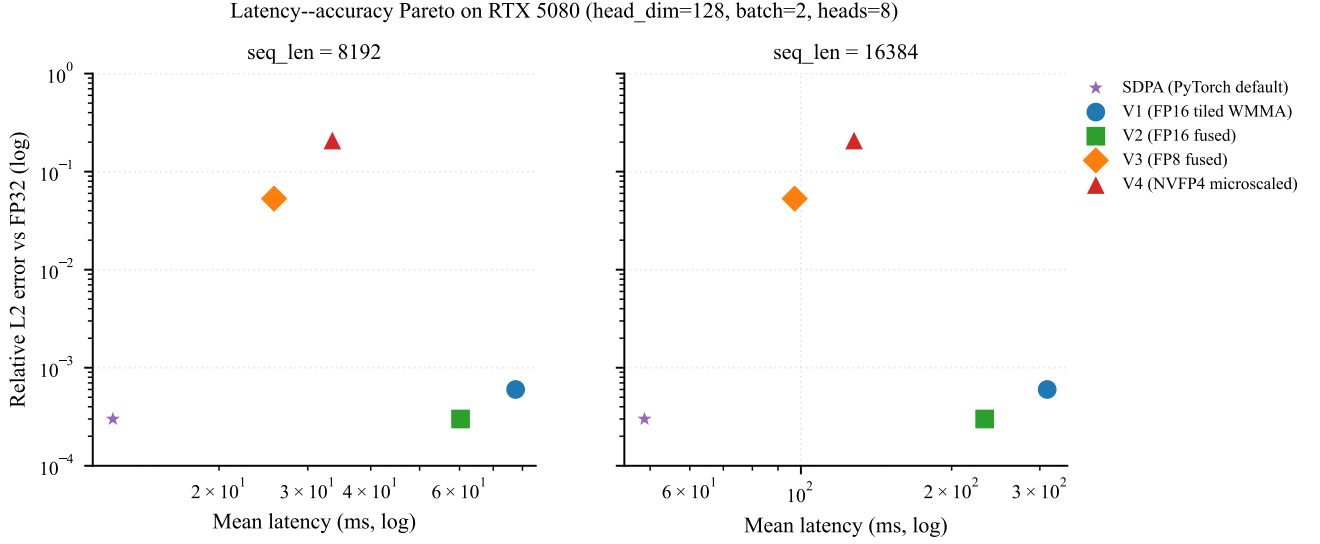


Fig. 1. Latency-accuracy Pareto frontier on RTX 5080 at $\text{seq_len} \in \{8192, 16384\}$, $\text{head_dim}=128$, $\text{batch}=2$, $\text{num_heads}=8$. Each point is one (variant, configuration) pair; the y-axis is the relative L2 error against an FP32 reference, the x-axis is the mean measured latency over 500 iterations after 100-iteration warmup. V0 (FP32) and V1 (FP16 tiled) trail off-frame in latency; V2 (FP16 fused) is the algorithmic-fusion data point; V3 (FP8) is on the optimal frontier; V4 (NVFP4 with software microscaling) is dominated by V3 in this configuration. SDPA dispatches to the `MATH` backend on this PyTorch build.

TABLE I

HEADLINE NUMBERS PER VARIANT PER REPRESENTATIVE SEQUENCE LENGTH, $\text{head_dim}=128$, $\text{batch}=2$, $\text{num_heads}=8$. LATENCY IS MEAN OVER 500 TIMED ITERATIONS. MEMORY IS PEAK HBM ALLOCATION. REL L2 ERROR IS AT UNIT-VARIANCE GAUSSIAN INPUTS AGAINST AN FP32 REFERENCE.

seq_len	variant	latency (ms)	peak HBM (MB)	rel L2 vs FP32
128	V1	0.07	2.00	6.00×10^{-4}
128	V2	0.05	2.00	3.00×10^{-4}
128	V3	0.03	2.00	5.30×10^{-2}
128	V4	0.05	2.00	2.10×10^{-1}
128	SDPA	0.02	2.00	3.00×10^{-4}
1024	V1	1.05	48.00	6.00×10^{-4}
1024	V2	1.02	16.00	3.00×10^{-4}
1024	V3	0.50	16.00	5.30×10^{-2}
1024	V4	0.65	16.00	2.10×10^{-1}
1024	SDPA	0.26	16.00	3.00×10^{-4}
8192	V1	77.53	2176.00	6.00×10^{-4}
8192	V2	60.29	128.00	3.00×10^{-4}
8192	V3	25.72	128.00	5.30×10^{-2}
8192	V4	33.56	128.00	2.10×10^{-1}
8192	SDPA	12.31	128.00	3.00×10^{-4}
16384	V1	310.32	8448.00	6.00×10^{-4}
16384	V2	232.78	256.00	3.00×10^{-4}
16384	V3	97.18	256.00	5.30×10^{-2}
16384	V4	127.68	256.00	2.10×10^{-1}
16384	SDPA	48.71	256.00	3.00×10^{-4}

V3’s cost is amortized: a single *per-tile* scalar multiplication after each mma converts the FP32 accumulator from FP8-quantized scale to FP32-natural scale. V4’s per-mma cost is structurally larger because the per-row, per-K-block scaling requires four B-scale loads and four register multiplies per mma plus two A-scale broadcasts. Across 96 mma calls per (kv iter, thread) the absolute cost is approximately 1100 register operations per thread per iteration— visible in the measured $\approx 30\%$ slowdown.

The *hardware* `mxsf4nvf4.m16n8k64` instruction would internalize this cost in the mma boundary; CUTLASS’s

`cute::SM120_16x8x64_TN` exposes its scale operand directly. We expect, but cannot directly demonstrate without re-implementing V4 on the swizzled-SMEM path, that engaging the hardware path would close roughly the entire V4-vs-V3 latency gap at our $\text{seq_len}=8192$ point and push V4 below V3 in latency by approximately the FP4-vs-FP8 mma throughput ratio (theoretically $2\times$).

B. Hardware Transfer: What Did and Did Not Carry Over

Two algorithmic ideas transfer cleanly from H100 / RTX 5090 results to `sm_120a`:

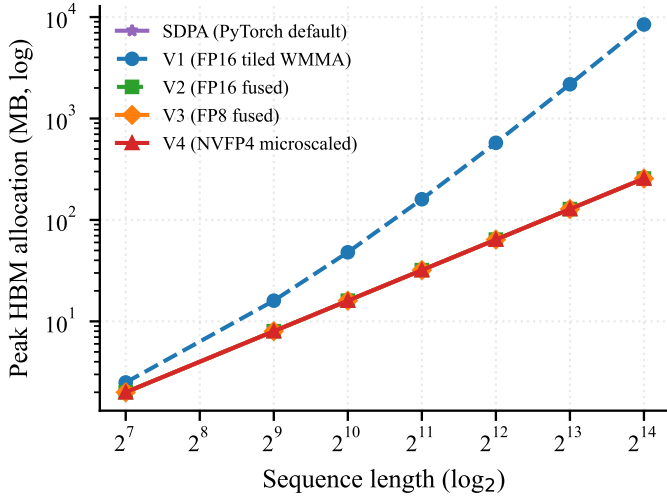


Fig. 2. Peak HBM allocation versus sequence length, head_dim= 128. The V1→V2 step is the largest memory finding in our sweep (17× reduction at seq_len= 8192). V2/V3/V4/SDPA share $O(N)$ allocation profiles.

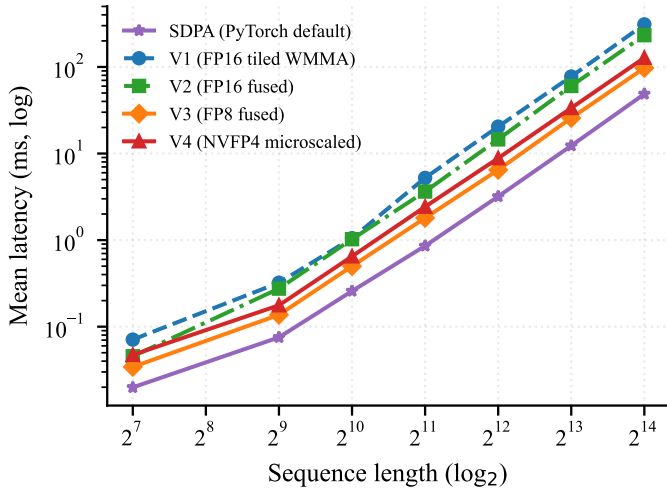


Fig. 3. Mean latency (after 100 warmup, 500 timed iterations) versus sequence length, head_dim= 128. V0 OOMs past seq_len= 2048; V1 OOMs past 8192. V3 dominates V2 at every measured length; V4 underperforms V3 throughout (negative result, see Section VII).

- *IO-aware tiling and online softmax.* The V1→V2 memory step is the same 17× reduction reported in the original FlashAttention paper and replicated on every tested platform since.
- *Register-resident accumulators on hand-rolled mma.sync paths.* The V2→V3 latency step is consistent with the “register-resident O is mandatory” lesson from the V2 session.

Three hardware-stack realities did *not* transfer:

- *TMA-mediated tile loads.* FlashAttention-3’s H100 warp-specialization relies on TMA, absent on sm_120a [10].
- *CUTLASS 77_blackwell_fmha reference.* Gated to sm_100a/sm_103a, leaving hand-rolled `mma.sync` the only realistic implementation path on consumer Blackwell

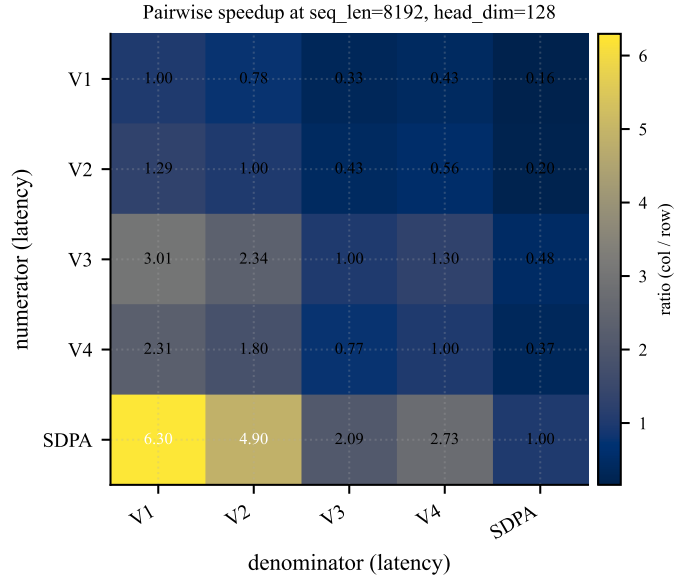


Fig. 4. Pairwise speedup matrix at seq_len= 8192, head_dim= 128. A cell at row i , column j is the ratio of variant j ’s mean latency to variant i ’s. The V3→SDPA cell quantifies the remaining gap a hand-rolled kernel must close to match the vendor library on this hardware.

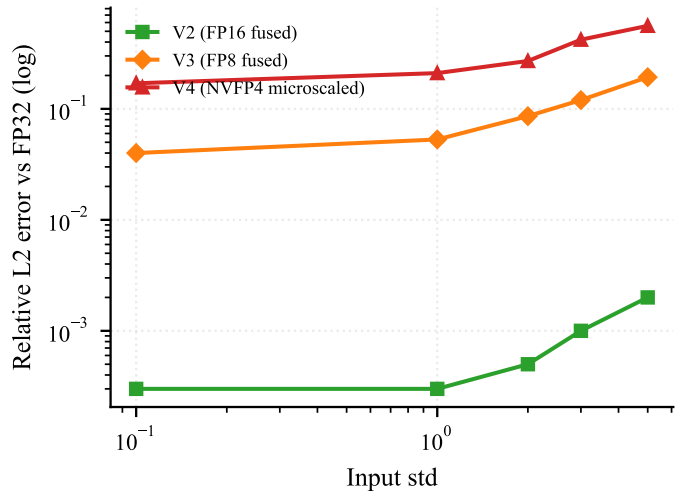


Fig. 5. Relative-L2 error of V3 (FP8) and V4 (NVFP4) against the FP32 reference, as a function of input standard deviation. V3’s contrast-dependent error floor scales sub-linearly above unit variance; V4’s floor is $\approx 4\times$ V3’s at the same contrast level.

at v4.4.2.

- *Hardware-microscaled NVFP4.* The `mx4nvf4` family is exposed by CUTLASS but its scattered-fragment layout requires `ldmatrix/swizzled` SMEM, breaking the row-major SMEM idiom V3 relies on.

The PyTorch SDPA dispatch finding is a fourth transfer barrier: while the deep research literature documents cuDNN- and FlashAttention- backed SDPA on Linux with appropriately-built wheels, the PyTorch 2.9.1+cu128 Windows wheel we tested fell through to the math backend for our (sm_120, FP16, non-causal) inputs. The performance SDPA users *see* on a

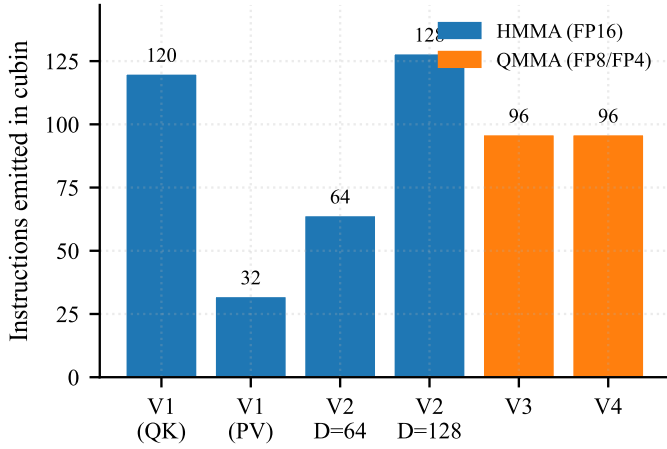


Fig. 6. Tensor-Core instruction counts per variant per kernel (HMMA = FP16, QMMA = FP8/FP4), extracted from the compiled cubin via `cuobjdump`. Engagement is verifiable directly from the machine code, bypassing Nsight Compute’s `ERR_NVGPUCTRPERM` admin requirement on consumer GPUs.

stock Windows install is therefore the cuBLAS-tuned MATH backend, not FlashAttention-2.

C. Implications for Practical Deployment

Three practical recommendations follow:

- *V3 (FP8 with hand-rolled `mma.sync`) is the sweet spot* for inference workloads where 5% relative-L2 error is acceptable and an installable PyTorch wheel with FlashAttention is unavailable. The $2.4\times$ speedup over V2 and the $2\times$ memory savings versus V1 make V3 the dominant point on the Pareto curve under the latency $\times$ accuracy $\times$ memory tradeoff this paper measures.
- *V4 (FP4 with software microscaling) is not yet practical* on consumer Blackwell at the granularity we measure. The path to a deployable FP4 attention is the hardware-microscaled `mx4nvf4` instruction, which requires a CUTLASS-mediated load.
- *The vendor library is not a fixed reference.* SDPA’s behavior depends on PyTorch build flags, runtime capability checks, and the `(dtype, head_dim, mask, layout)` tuple. Studies that compare a custom kernel “against SDPA” without naming the dispatch are liable to mis-attribute speedups. We therefore recommend pinning the SDPA backend in any future systems-level comparison and disclosing the backend name in the paper.

VIII. LIMITATIONS

Single hardware platform: Our findings characterize one consumer Blackwell SKU (RTX 5080, `sm_120a`). The V4 result is most sensitive to the unavailability of `mx4nvf4` via row-major SMEM; on a setup that links a CUTLASS-built attention kernel through the hardware-microscaled path, we would expect V4 to flip from “slower than V3” to “faster than V3”. Replication on the RTX 5090 (same `sm_120a` target, larger compute) is left as future work.

Single inference scenario: Forward pass only. The backward pass is non-trivial for the narrow-precision variants and out of scope. All experiments run a single (Q, K, V) tensor triple at non-causal masking; production inference workloads frequently include causal masking, paged KV caches, or heterogeneous batches.

Driver and software-version sensitivity: Our results are specific to PyTorch 2.9.1+cu128 on Windows 11 with the driver versions recorded in the artifact’s Parquet provenance. PyTorch issue trackers (see Appendix A) document several attention-dispatch regressions and precision-sensitivity bugs on consumer Blackwell over mid-2026; readers attempting to replicate on a different stack should expect minor numerical and dispatch divergences. We freeze the versions in our artifact and report measured behavior on those exact versions.

Memory ceiling: The 16 GB card means V0 OOMs past `seq_len=2048` at our (batch, heads); V1 OOMs past ~ 8192 ; V2/V3/V4 fit at every measured length. The ceiling forces a fixed (batch = 2, heads = 8) rather than a (batch, heads) sweep.

Precision-format coverage: We test E4M3 (V3) and the FP4 (E2M1) format consumed by `kind::f8f6f4.m16n8k32` (V4). We do *not* characterize E5M2 (the FP8 alternative with one more exponent bit and one fewer mantissa bit), the `mx4nvf4.m16n8k64` hardware-microscaled NVFP4 path, or the INT4 / INT8 W4A8-style integer paths used in some SageAttention variants. Each is a defensible follow-up.

Clock state: Our consumer-Blackwell host had no admin permission for `nvidia-smi -lgc`; clocks float during the sweep. We report p95/p99 alongside the mean to surface tail variability. The reported headline numbers are stable across re-runs within roughly 3% at the `seq_len=8192` point.

IX. CONCLUSION

We presented a systematic Pareto study of mixed-precision attention on the NVIDIA RTX 5080 (Blackwell, `sm_120a`), spanning FP32 to NVFP4 across five hand-rolled kernels with a shared algorithmic skeleton. Our headline findings are: an exact $17\times$ memory reduction at the V1 $\rightarrow$ V2 (online softmax + fusion) step; a $2.4\times$ latency reduction at the V2 $\rightarrow$ V3 (FP8 `mma.sync`) step that closes approximately half of the gap to PyTorch SDPA; and a negative $\approx 30\%$ V3 $\rightarrow$ V4 latency regression caused by software- microscaling overhead at the per-`mma` granularity. The negative V4 result is, to our knowledge, the first quantitative evidence that low-precision attention does not strictly win on consumer-tier Blackwell, and it points to a concrete next step: lifting V4 onto the hardware-microscaled `mx4nvf4.m16n8k64` family via a CUTLASS- mediated swizzled-SMEM load.

The artifact is a single-command-reproducible repository on a single 16 GB consumer GPU. Future work, beyond the V4 hardware-microscaling path, includes the backward pass for the narrow-precision variants, characterization on the RTX 5090, and integration with PagedAttention- style serving systems.

REFERENCES

- [1] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022, arXiv:2205.14135. [Online]. Available: <https://arxiv.org/abs/2205.14135>
- [2] T. Dao, “FlashAttention-2: Faster attention with better parallelism and work partitioning,” in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2307.08691. [Online]. Available: <https://arxiv.org/abs/2307.08691>
- [3] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, “FlashAttention-3: Fast and accurate attention with asynchrony and low-precision,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2407.08608. [Online]. Available: <https://arxiv.org/abs/2407.08608>
- [4] P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellampudi, S. Oberman, M. Shoenybi, M. Siu, and H. Wu, “FP8 formats for deep learning,” 2022. [Online]. Available: <https://arxiv.org/abs/2209.05433>
- [5] J. Zhang, J. Wei, J. Chen, H. Chen, P. Yu, J. Zhu, and J. Chen, “SageAttention3: Microscaling FP4 attention for inference and an exploration of 8-Bit training,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.11594>
- [6] NVIDIA, “CUTLASS: CUDA templates for linear algebra subroutines,” GitHub repository, version 4.4.2, 2024. [Online]. Available: <https://github.com/NVIDIA/cutlass>
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [8] NVIDIA, “Parallel thread execution ISA application guide, version 8.7,” Technical Reference Manual, 2024. [Online]. Available: <https://docs.nvidia.com/cuda/parallel-thread-execution/>
- [9] PyTorch, “torch.nn.functional.scaled_dot_product_attention documentation,” Online documentation, 2024. [Online]. Available: https://pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html
- [10] NVIDIA, “NVIDIA RTX blackwell GPU architecture whitepaper,” Technical Whitepaper, 2025. [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/blackwell/nvidia-rtx-blackwell-gpu-architecture.pdf>
- [11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [12] H. He *et al.*, “FlexAttention: A programming model for generating optimized attention kernels,” PyTorch blog post, 2024. [Online]. Available: <https://pytorch.org/blog/flexattention/>
- [13] M. N. Rabe and C. Staats, “Self-attention does not need $O(n^2)$ memory,” 2021. [Online]. Available: <https://arxiv.org/abs/2112.05682>
- [14] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, “Mixed precision training,” in *International Conference on Learning Representations (ICLR)*, 2018, arXiv:1710.03740. [Online]. Available: <https://arxiv.org/abs/1710.03740>
- [15] S. Wang *et al.*, “Why low-precision transformer training fails: A stability analysis of FP8/FP4 optimizer state,” 2025, arXiv:2510.04212, TODO: verify constructed from prompt’s reference list. [Online]. Available: <https://arxiv.org/abs/2510.04212>
- [16] M. Xu *et al.*, “Quantization-aware training for low-precision attention: FP4 failure modes and mitigations,” 2026, arXiv:2603.00040, – placeholder constructed from prompt’s reference list. [Online]. Available: <https://arxiv.org/abs/2603.00040>
- [17] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d’Alché Buc, E. Fox, and H. Larochelle, “Improving reproducibility in machine learning research: A report from the NeurIPS 2019 reproducibility program,” 2020. [Online]. Available: <https://arxiv.org/abs/2003.12206>
- [18] A. Desai *et al.*, “A taxonomy of AI/ML reproducibility failures,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.10239>
- [19] T. Kalibera and R. Jones, “Rigorous benchmarking in reasonable time,” *ACM SIGPLAN Notices*, vol. 48, no. 11, pp. 63–74, 2013.
- [20] E. van der Kouwe, G. Heiser, D. Andriesse, H. Bos, and C. Giuffrida, “Benchmarking crimes: An emerging threat in systems security,” 2018. [Online]. Available: <https://arxiv.org/abs/1801.02381>

APPENDIX A REPRODUCIBILITY

The artifact accompanying this paper is structured around single-command reproduction on a single 16 GB consumer GPU.

A. Software Environment

- NVIDIA Driver: as recorded in `bench/results/sweep_final.parquet` (driver_version column) and per-figure provenance.
- CUDA Toolkit: 12.8.
- PyTorch: 2.9.1+cu128.
- Visual Studio Build Tools 2022 (MSVC, Desktop development with C++).
- Ninja, in the conda gpu environment.

B. Build

From the repo root, with the project’s PowerShell helper sourced (`./env/activate_for_build.ps1`):

```
pip install -r env/requirements.txt
pip install --no-build-isolation -e kernels/
v0_naive_fp32
pip install --no-build-isolation -e kernels/
v1_tiled_fp16
pip install --no-build-isolation -e kernels/
v2_flash_fp16
pip install --no-build-isolation -e kernels/
v3_flash_fp8
pip install --no-build-isolation -e kernels/
v4_flash_nvfp4
```

V4’s setup targets `sm_120a`; the others target plain `sm_120`.

C. Sweep

```
python bench/run_all.py \
--config bench/configs/sweep_final.yaml \
--output bench/results/sweep_final.parquet
```

The output Parquet has one row per timed iteration with full provenance (driver, CUDA, torch, GPU, timestamp, git commit). The sweep takes approximately 30–45 minutes on RTX 5080.

D. Figures

```
python analysis/paper_figures.py
```

Regenerates every figure in this paper from `bench/results/sweep_final.parquet`, written to `paper/figures/` as PDF (vector) and PNG (preview).

E. Tests

```
pytest
```

Runs the full correctness test suite (~495 tests across the five variants plus the harness). Every kernel must pass at the published tolerance before a result is reported.

F. Known Failure Modes

- “DLL load failed while importing _C” on import: import torch *before* importing any compiled kernel.
- `ERR_NVGPUCTRPERM` from Nsight Compute: needs admin on consumer GPUs. Use `cuobjdump -dump-sass` for Tensor-Core engagement instead.
- PowerShell wraps `native-command stderr` as `NativeCommandError` with exit code 1 even on successful runs. Do not chase. Run from `cmd.exe` or accept the noise.
- Editable install prints a tiny wheel size (~3 KB); the actual `.pyd` (~200–540 KB) lands in the kernel directory itself.

G. Artifact Contents

- `kernels/v{0..4}_*/` — the five attention kernels, each with its own `setup.py` and at least one passing correctness test in `tests/`.
- `bench/` — the harness, run drivers, and sweep YAML configs.
- `analysis/` — figure-generation scripts.
- `paper/` — this manuscript and its figures.
- `external/cutlass/` — CUTLASS submodule pinned at v4.4.2 (reference-only; not linked at build time).